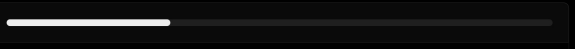
Modal
Multi Select
Note
Pagination
Phone
Pie Gauge
Pill
Progress
Project Banner
Radio
Relative Time Card
Scroller
Search Input
Select
Separator
Sheet
Show more
Skeleton
Slider
Snippet
Spinner
Split Button
Status Dot
Switch
Table
Tabs
Text With Copy Button
Textarea

Progress

Display progress relative to a limit or related to a task.

Default

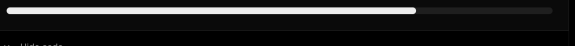


Hide code

```tsx
import { Progress } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return <Progress value={30} />;
}
```

Custom max



Hide code

```tsx
import { Progress } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return <Progress max={40} value={30} />;
}
```

Dynamic colors



Hide code

```tsx
import { Button, Progress } from '@vercel/geistcn/components';
import { useState, type JSX } from 'react';

export function Component(): JSX.Element {
  const [value, setValue] = useState(0);

  return (
    <div className="flex flex-col items-start justify-start gap-6 flex-initial">
      <Progress
        colors={{
          0: 'var(--geist-foreground)',
          25: 'var(--geist-error)',
          50: 'var(--geist-warning)',
          75: 'var(--geist-highlight-pink)',
          100: 'var(--geist-success)',
        }}
        value={value}
      />
      <div className="flex flex-row items-stretch justify-start gap-4 flex-initial">
        <Button
          onClick={() => {
            if (value < 100) setValue(value + 10);
          }}
          size="small"
        >
          Increase
        </Button>

        <Button
          onClick={() => {
            if (value > 0) setValue(value - 10);
          }}
          size="small"
          variant="secondary"
        >
          Decrease
        </Button>
      </div>
    </div>
  );
}
```

Themed

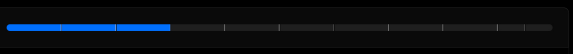


Hide code

```tsx
import { Progress } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <div className="flex flex-col items-stretch justify-start gap-6 flex-initial">
      <Progress type="success" value={100} />
      <Progress type="error" value={10} />
      <Progress type="warning" value={40} />
      <Progress type="secondary" value={70} />
    </div>
  );
}
```

With Stops

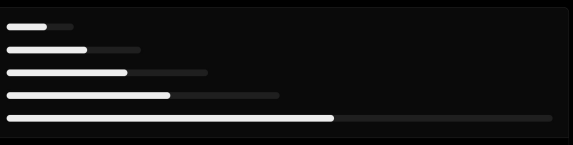


Hide code

```tsx
import { Progress } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <Progress
      value={30}
      type="success"
      stops={[
        { value: 10, tooltip: '10%' },
        { value: 20, tooltip: '20%' },
        { value: 30, tooltip: '30%' },
        { value: 40, tooltip: '40%' },
        { value: 50, tooltip: '50%' },
        { value: 60, tooltip: '60%' },
        { value: 70, tooltip: '70%' },
        { value: 80, tooltip: '80%' },
        { value: 90, tooltip: '90%' },
        { value: 95, tooltip: '95%' },
      ]}
    />
  );
}
```

Widths



Hide code

```tsx
import { Progress } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <div className="flex flex-col items-stretch justify-start gap-6 flex-initial">
      <Progress value={60} width={100} />
      <Progress value={60} width={200} />
      <Progress value={60} width={300} />
      <Progress value={60} width="50%" />
      <Progress value={60} width="100%" />
    </div>
  );
}
```

Heights



Hide code

```tsx
import { Progress } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <div className="flex flex-col items-stretch justify-start gap-6 flex-initial max-w-prose">
      <Progress value={60} height={4} />
      <Progress value={60} height={10} />
      <Progress value={60} height={50} />
      <Progress value={60} height={200} />
    </div>
  );
}
```

Best Practices

When to use

- Determinate work whose total is knowable, like file uploads, multi-step setup, build steps, or batch deletions.
- For short indeterminate waits (~1–3s), use **Spinner**; for inline copy like `Saving`, use **Loading Dots**.
- For usage against a quota or ratio, use **Gauge**; the circle reads as health, the bar reads as progress.

Behavior

- Use `max` for the real ceiling (`max={files.length}`), not a hardcoded `100`. The component computes the percentage from `value / max`.
- Threshold colors via `dynamic colors` should mirror the same breakpoints used elsewhere (warning at the same threshold a quota note fires).
- Use stops for genuine multi-stage work and label the stage next to the bar (`Step 2 of 4 · Building`); a stop with no label is decorative noise.

Content

- Pair the bar with text naming the work and units: `Uploading 12 of 30 files`, `Building · 1.2 GB / 4 GB`. The bar alone doesn't say what's progressing.
- Don't append `successfully` or `complete` once the bar fills; swap to a completion state (toast, success row, redirect).
- For long operations, name the work in the surrounding copy (`Building deployment…`) instead of leaving a bare

percentage.

Accessibility

- Component sets `role="progressbar"` with `aria-valuemin`, `aria-valuemax`, and `aria-valuenow`; pass an accessible name through `aria-label` on the wrapper or a sibling `<label>` tied with `aria-labelledby`.
- Throttle `aria-valuenow` updates to roughly once a second so screen readers don't announce every increment of a fast upload.
- Stops accept their own `ariaLabel`; name each one (`Build complete`, `Tests complete`) so the bar is navigable without sight.